Némoz Burkhalter
#12-135542 / nemoz.burkhalter@gmail.com

BACHELOR OF SCIENCE
Games Programming

Situational Evaluation of Goal Oriented action planning

What are the tradeoffs of GOAP compared to the other standard AIs of the game industry?

BACHELOR THESIS - 6GSC0XF101.2
Supervisors: Serge Djoungong, Elias Farhan

SAE Institute Geneva October 10th, 2026
9609 Words

I, Némoz Burkhalter, certify having personally written this Bachelor thesis.

I also certify that I have not resorted to plagiarism and have conscientiously and clearly mentioned all borrowings from others.

Geneva / 10.10.2026 / signature :



Table of Contents

Abstract 6

1 Introduction 7

2 State of the art 7

- Hard-coded behaviors 8
- FSM and behavior trees 8
- GOAP 10
  - Definitions 10
  - Planning 10
- HTN 10
- Summary 11

Qualitative analysis 11

- Analysis of Works 11
- Analysis of F.E.A.R. 12
  - Architecture 12
  - Trade-offs of F.E.A.R.'s AI 14
- A look at Shadow of Mordor 15
- A look at Tomb Raider 16
- Expert interview: Jeff Orkin 18

Media Project 19

- Project scope and goals 19
- Software used 19
- Detailed project plan 20
  - Simulation description 20
  - Step 1: Implementing a GOAP planner 20
  - Step 2: Environment 20
  - Step 3: Implementation of actions and execution 21
  - Step 4: Conduct experiments 21
  - Step 5: Analysis 21
- Project creation 21

Overview of project implementation 22
GOAP WorldState 22
GOAP Actions 24
GOAP Planner 24
IDA* Graph 28
EnemyAI 28
Game Director 28
Testing Protocol 28
Time 28
Memory usage 28
Behavior 29
Versatility 29
Testing 29
Single Action 29
Setup 29
Results 30
Multiple Actions 30
Setup 30
Results 30
Long plan 30
Setup 30
Results 30
Change of situation 31
Setup 31
Analysis of results 31
Conclusion 32
Quantitative analysis 33
Expected results 33
Values Observed 33
Limitations 33
Conclusion 34
Bibliography 36

Ludography 39
List of Figures 39
Appendix 40
Interview with expert : Jeff Orkin 40

Abstract

This thesis investigates the trade-offs of Goal-Oriented Action Planning (GOAP) compared with common video game AI techniques, particularly Finite-State Machines (FSM). Through an analysis of existing games and the implementation of a GOAP planner using IDA*, this research compares behavioral adaptability, versatility from a production standpoint and computational costs. The results show that GOAP can be used to create more context-dependent action sequences and emergent behavior. The results also show that GOAP systems require greater computational resources and are not necessarily easy to work with for a production team. The thesis concludes that the choice of technique depends on the complexity of the desired behaviors and the constraints of the game.

1 Introduction

Video games present interactable worlds where players engage with the fantasy presented. While graphics, sound design, controls and other systems are quite important to experience proposed by a game, this thesis will focus on the world's simulation. Specifically, the use of Artificial Intelligence (AI) to simulate characters, armies, cities and other decision-making entities that are involved in games.

From Pong (Atari Inc.) in 1972 to ***Middle-earth: Shadow of Mordor*** (Warners Bros. Interactive Entertainment) in 2014 AI-driven agents have evolved alongside video games. As games became more complex the need for more complex and varied Ais grew stronger. There now exists a great number of techniques to simulate agents.

In this thesis, the author will focus on one such technique, Goal Oriented Action Planning (GOAP) a technique which applies graph navigation algorithms to decision making to achieve a specified goal.

The thesis will investigate the characteristics of Goal Oriented Action Planning and draw comparisons with other AI techniques such as hard-coded behaviours, Hierarchical Task Network (HTN) and Finite-State Machines (FSM) which will be expanded upon in 3. Qualitative Analysis.

To investigate these techniques the author will implement a simple GOAP planner using the recommended industry practices. This implementation will then be the basis of an analysis of the behaviour of agents using GOAP. These results will then be compared and contrasted to other industry-standard practices. Additional factors will include the ease of use to express behaviours from a developer's standpoint, the perceived intelligence of the agents and the computation cost.

2 State of the art

In this thesis, AI will be used with the definition of a decision-making algorithm in the context of video games. While other types of AIs such as generative AIs and LLMs (Large language Models) are beginning to be used in video games production, they are outside the scope of this thesis. The focus of this paper will be on the use of the AI for Non-Player Characters (NPCs) which is the most common usage in the industry **(**Thompson Tommy, 2024, video essay**)**.

The various uses of AIs

First and foremost, video games AIs are used in the context of a simulated world and serve to enrich the ways that the simulation responds to the player's actions.

For instance, in action games such as FPS (First-Person Shooters), AI opponents are the main challenge offered to the player. An enemy has multiple actions available to them, they can move, shoot the player, go to cover, throw explosives or coordinate with other enemies. In a strategy game, an agent will decide how to use its resources when and how to engage in conflicts. Resolving which action is most pertinent is the purpose of the AI. The

relevance of each action will depend upon the desired game experience and the technical constraints. A more challenging AI is one that will choose actions that are more efficient at defeating the player, in this view it is an optimization problem. Although the designers of a game will often choose to adjust the difficulty by having AIs take less optimal options **(CITATION https://www.researchgate.net/publication/390557730_AI_in_Gaming_Procedural_Content_Generation_NPC_Behavior_and_Real-Time_Strategy).**

Agents are also used to teach the player in an implicit manner. In Mewgenics (2026), its creator Edmund McMillen explains that the AI enemies often demonstrate complex interactions between the game's systems that the player would not typically stumble unto by themselves **(CITATION https://www.youtube.com/watch?v=rVNQ4jh4lTw).** AIs

Technical limitations present another challenge for AI designs. Video Games are real time applications which cannot always afford long computation times when an NPC need to decide what to do next.

There exists a multitude of techniques to drive agents' behaviour each with their own advantages and disadvantages. Ideally, as a tool for game designers and an element of gameplay for the players, AIs should be easy to design for and be interesting to witness and interact with when applicable.

Hard-coded behaviors

One of the simplest ways to control an agent is to implement its behaviour directly through conditional statements and predefined rules. For example, in Pong, the opposing paddle moves at a given speed to match the ball's y position.

Such approaches are straightforward to implement and can be effective when the number of possible situations is limited. Their behaviour is predictable, and developers retain direct control over the agent's responses. However, as the number of actions and possible situations increases, the number of conditions and interactions that must be managed grows too. Extending behaviors may require modifying existing rules while handling precise edge cases.

More structured approaches seek to organize this decision logic into representations that are easier to understand, extend and maintain. Finite-state machines and behaviour trees are two of the most common examples.

FSM and behavior trees

A Finite-State Machine (FSM) is an abstraction that postulates that an agent has a finite number of states it can be in where states represent the current mode of behavior and the available actions. An example would be a sniper being set up as opposed to on the move or reloading. Guards can be on their patrol routine or in chase mode.

An agent can transition from one state to another whether through internal or external stimuli such as its health going under a threshold. The FSM will then use state-dependent logic to decide what action to take.

FSM is a relatively simple system to implement and makes clear how the agent operates. This makes them well suited for games that focus on enemy patterns or that have well-defined states. Hollow Knight (Team Cherry 2017) uses such a system to make boss fights predictable and learnable encounters. As the player grows more familiar with the boss's behavior, they find more opportunities to strike between or during attacks.

FSMs are computationally quite fast since they perform a single evaluation of the current state to decide on an action. But, when the number of states, actions and conditions for those actions grows, FSMs become somewhat unwieldy **(CITATION NEEDED)** to design with.

Behavior trees are quite similar to FSMs where the actions available depend on what the agent is currently doing and actions can lead to other actions or change the current state. Although behavior trees express actions and states like an FSM, they provide another way to organize decision-making. Rather than representing behaviour primarily through transitions between states, they organize conditions and actions as nodes in a hierarchical tree. There are no explicit state in behavior tress only actions and their relations to each other.

The typical nodes are Sequence node which perform all nodes under it in sequence and the Selector node which conditionally jump to one of its child nodes. Behaviour trees became prominent in game AI partly because their hierarchical structure can make complex decision logic more modular.

While these methods work for all types of agents and can create very complex behaviors as discussed by lovino et. al (2022) **(CITATION https://www.sciencedirect.com/science/article/pii/S0921889022000513 154)**, they still require the developers to define those behaviors, creative interactions are defined explicitly.

Ideally, as a tool for game designers and an element of gameplay for the players, AIs should be easy to design and be interesting to witness and interact with when applicable. In a way, the goal of AIs is to populate the game world to make it richer and more immersive for the player. Comparing AIs and players can lead to interesting insight. Indeed, most AIs in third person shooters have the same action verbs as the player, controlling a character that can move, sometimes jump, aim and shoot. They have a health resource and can die. They are trying to help the player character or to kill them. In strategy games like sid meier's civilisation games, the opponent AI can perform all the same actions as the player (though sometimes it receives bonus resources or penalties for balancing purposes according to the difficulty setting).

Final-state machines must be given strategies via explicit actions to take in given situations or by calculations of available resources and data on their environment. But players can simply adapt to situations using their previous knowledge. When a player unlocks a new unit, they can think about how that unit integrates with their strategy and think about how to use it. When a new system is introduced like flammable oil, Finite-State Machines must be given a clear definition of how to use and interact with the new system, it is the role of the designer to decide on a plan for the agent. This constraint motivates, in part, the next algorithm: Goal Oriented Action Planning.

GOAP

Goal Oriented Action Planning (GOAP) is an approach to game AI where the agent will be given a goal and a set of all actions it can perform. Using those actions the agent will formulate a plan that reaches its goal.

This approach first appeared commercially in F.E.A.R. in 2005, an FPS made by Monolith Productions. Jeff Orkin, lead AI developer on this game writes in his 2004 paper "Applying Goal-Oriented Action Planning to Games" about the benefits of GOAP stating that the technique provides more varied behaviors compared to more traditional techniques as the agent adapt to new situations. **(CITATION)**. Furthermore, it simplifies changing designs as the decision logic is largely divorced from the game world.

Definitions

World State: A GOAP implementation represents everything relevant to the agent as a world state. (Pittman David, 2007, 12-19**)(CITATION**).

Goal: The goal of the agent is defined as a desired world state.

Actions: Actions are defined as how they modify a world state and the conditions necessary for the action to be performed. Actions can have no conditions or have multiple conditions which are typically represented as a world state or sub-goal. They must also have a cost, the use of which is for the planning algorithm.

Plan: A plan is a sequence of action which modifies the current world state in such a way that it satisfies the goal.

Planning

To plan for actions, Goal Oriented Action Planning considers the world state as a node in a graph. Actions act as transitions from one node to another (example: worldstate {hasPizza: TRUE} becomes {hasPizza: FALSE} after performing the action EatPizza). The complete graph represents all possible courses of action. As such we can use pathfinding algorithms to navigate this graph. One such algorithm is A*, a standard pathfinding algorithm in video games, which guarantees to find the most optimal path (provided the heuristic function is acceptable) and does not require to explore the entire graph to do so.

While Jeff Orkins initially uses A* for planning, Ian Milligton recommends in *Artificial Intelligence for Games* (Millington Ian 2006, p.394) to use IDA* (Iterative Deepening A*). IDA* performs a depth first search while keeping the heuristic metric of A* which allows it to use less space in memory but can mean revisiting the same node multiple times being potentially slower.

HTN

Hierarchical Task Network (HTN) planning is another approach to automated planning. Where GOAP finds a sequence of actions based on those action's conditions and

effects in relation to the goal, HTN defines actions at various levels of complexity. When the agent needs to perform an action, it is repeatedly decomposed into a sequence of defined actions which are of a smaller scope until the actions are no longer abstractions.

For example, a high-level task such as *Defeat Opponent* could be decomposed into subtasks such as *Prepare Equipment*, *Reach Suitable Position*, and *Engage Opponent*. These tasks may themselves be decomposed until the resulting sequence consists solely of executable actions. The decomposition methods encode knowledge about how a task can be accomplished.

GOAP and HTN planning therefore differ in how they represent and organize the planning problem. GOAP commonly searches for sequences based on action preconditions, effects, and desired world-state conditions, whereas HTN planning relies on decomposing tasks through predefined methods. Both generate sequences of actions, but their respective strengths depend on the problem. HTN will need the developer to specify how each action can be broken down.

Summary

The techniques presented in this chapter all allow for agents to decide in real-time how they will behave. However, each approach involves trade-offs both during production and during gameplay. Where hard coding an FSM provide predictable and clear behaviors, HTN and GOAP can be unpredictable. This unpredictability can be desirable as it allows for the emergence of unplanned and interesting behavior.

While all of these systems can be used for all cases, the amount of work needed to achieve a given result will vary greatly. Implementing a new action in an FSM requires to defines every situation it should be used in while a planner using GOAP will figure out when to use that action based solely on what it does and requires. Conversely creating a specific behavior such as a skittish enemy must be done indirectly in GOAP.

3 Qualitative analysis

Analysis of Works

To better understand how GOAP works in commercial games and applications, the author will examine F.E.A.R. where the technique was first popularized. Particular attention will be given to the advantages afforded by the technique and how the disadvantages were addressed or covered. Throughout, the advantages and disadvantages of GOAP will be compared to what an FSM would offer or cost.

Finally, to further comprehend GOAP’s place in the industry, the analysis will use multiple games as examples of how GOAP implementations differ.

Analysis of F.E.A.R.

Architecture

F.E.A.R. of First Encounter Assault Recon is an action-packed game where the player is part of a special force which meets a paranormal threat which involve clones of the special forces.

Through its usage of GOAP, the game was praised for its great AI in reviews Jason Ocampo writes for GameSpot: “Put simply, these are the smartest, most aggressive, most tactically oriented AI opponents that we’ve ever encountered in a shooter, and they’re downright impressive.” (Ocampo Jason, 2005).

Like most games of the time, F.E.A.R. uses an FSM as the basis for its AI (Orkin Jeff, 2006, p.1). However, this FSM only uses three basic states Goto, Animate and Use smart object

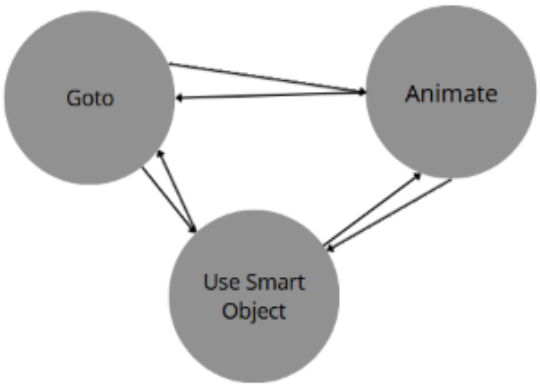


Figure 1 F.E.A.R.’s Finite State machine (Adapted from Orkin Jeff in Three States and a Plan: The A.I. of F.E.A.R. 2006)

Where Animate is a versatile state as every action is an animation.

It was considered to embed in each state the logic to transition from one to the other, but they decided to instead use planning to handle this logic. A reason behind this decision is to separate the designers from the scripting of the logic. Agents use cover and other environment features autonomously to achieve their goals. Additionally, there was only one AI programmer on the team and detailing all this logic would be an inefficient of his limited time (Orkin Jeff, 2006, p.3 and p.6).

This early version of GOAP builds upon STRIPS planning (*Stanford Research Institute problem solver)* but differs in its world state representation, in the addition of a cost for actions and conditions and effects (Orkin jeff, 2004). All of these features define GOAP and are still used twenty years later (example: Multi-Agent Performance Using Goal-Oriented Action Planning (GOAP) in a Procedural Game, Sjöblom Marcus, Svala Rasmus et al., 2025)

More specifically, the world state is represented in C++ (language used in F.E.A.R. for its AI logic) by a fixed size array of key/pair values. The values are boolean values (Example: TargetDead TRUE) or pointers to a value somewhere else in memory (Orkin Jeff, 2006, p.12). This limitation was likely a result of how planning uses a sizeable amount of memory space

per agent and access it often during planning (Long Edmund, 2007, p.54) and significant care had to be taken to optimize the memory accesses (Orkin Jeff, 2024, on youtube in *F.E.A.R. – The Retrospective* by AI and Games). For instance, the planner does not have knowledge of the current target as the fixed memory size did not allow for it, the target is supplied by an external director which decides which target is currently in focus.

All agents have a list of goals with a priority score attached to them. These goals can originate from the agents themselves (Example: GetToCover gets a higher priority as the enemy's health descends) or by a director which simulates squad behaviors distributing goals to available agents. This squad behavior is an unformalized version of STRIPS and HTN which the author does not detail and serves as a layer above GOAP.

An agent's decision is then the result of three layers of AI systems. A squad behavior decides what multiple agents should do and sets that as a goal. The agent will animate itself using the simple FSM whose actions will be overridden by the GOAP of the agent.

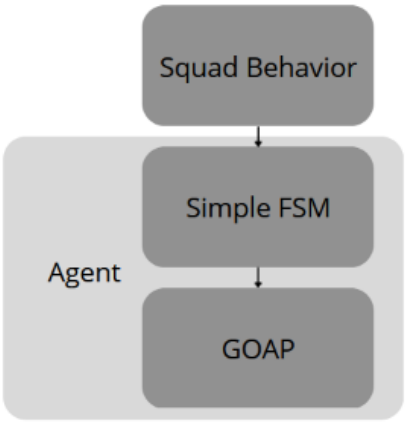


Figure 2 Author's depiction of F.E.A.R.'s layering of AI systems (2026)

The FSM can be largely ignored in the decision-making process of an agent as it mainly serves to animate the character based on what it is doing.

An interesting consequence of this compartmentation is that agents do not have knowledge of the other agents, collaboration only happens at the squad behavior level and is then communicated to the player through voice lines. Two agents moving together might have one ordering the other to move but this only happens at the squad level and neither agent has an impact on the other's decision making.

Actions are defined as in chapter 2 with effects and conditions, but the fixed size of the world state would not allow to evaluate complex conditions for the actions. The solution used was to use procedural conditions, actions are C++ classes that define world states as conditions and implement a CheckProceduralPreconditions() function to check conditions that can't be expressed in single values (example: checking if an escape path exists) (Orkin Jeff, 2006 p.12).

Plans were made relatively short by using actions that achieve a lot at once. For example, using a KillPlayer action rather than a simple Shoot action which would be repeated multiple times necessitating a longer sequence (Orkin, 2006)

GOAP's planning is done through A* which is possible through the introduction of costs to actions (as opposed to STRIPS actions). A* searches forward from the initial state to the goal state using actions as edges (Orkin Jeff, 2006, p.11). This search is made faster compared to an implementation closer to STRIPS thanks to the fixed size array of the world state which has an O(1) access time (Orkin Jeff, 2004) while STRIPS uses a linked list that has an O(n) access time complexity. However, like regular pathfinding, performing searches often is quite expensive and virtually all GOAP implementations avoid replanning frequently (Orkin Jeff, 2004) (Orkin Jeff et al., 2010) (Champandard Alex J., 2008).

Trade-offs of F.E.A.R.'s AI

As stated above, F.E.A.R. was a success and reviews praised the game's innovative and sharp AI and were considered the amongst the most advanced opponents at the time (Ocampo Jason, 2005). While the success of the game cannot be entirely attributed to its AI, its usage of GOAP distinguished the game from other shooters of the time (AI and Games, 2024).

On the production's side, GOAP was harder for the programmers to implement than FSM which was the main alternative (Orkin, 2006). However, the decoupling of the expression of actions and the planning of the actors made changing design requirements easier and more intuitive for designers (Orkin, 2008). For example, if the design changed to necessitate agents to light the room they are in if they are dark, it would only require adding a lighting action and a condition of the room being lit on relevant actions (Orkin, 2008). Where if the AI was wholly FSM, each state of the agent would need to have added logic for the lighting.

However, expressing specific behaviors was more difficult and less intuitive (Orkin, 2025, interview with Jeff Orkin, see appendix). This is to be expected as GOAP has an undirect relationship between the designed actions and the precise sequence of actions chosen. If a designer wants an enemy to be more aggressive or sheepish they can influence this behavior by tweaking action costs to change how often they are used. Whereas an implementation in FSM would allow the explicit nuance of detailing behaviors for each NPC type at the cost of being more complex to define.

One of the strengths of GOAP and therefore F.E.A.R., is the ability of the AI to combine actions to find context-specific plans which are credible even if the situation was unforeseen by the developers. This is a natural consequence of GOAP which automatically create emergent behaviors that the designer did not have to specify as a sequence (Pittman David, 2007, p.6). Nevertheless, GOAP's emergence is limited by the action set and how those actions interact with other systems. F.E.A.R. did not have many gameplay elements that the AIs could interact with outside of cover and the player. For instance, having explosive barrels, the effect of which would be to detonate and damage nearby things, if the shoot action is implemented with a procedural effect (which is supported in the implementation of the Action class) agents will automatically shoot the barrels when it is advantageous to do so to kill the player. Later GOAP implementations will incorporate more of these types of interplay between systems (Chris Conway et al., 2015, AI Summit).

While GOAP is a very important part of the AI of F.E.A.R., it is not the only one and the layered hierarchy allows for complex plans that happen over a longer term without

needing to explore an immense space of world states (Giuseppe Maggiore et al., 2013). However, this approach was not formalized at the time and was seen as a hybrid approach with HTN (Orkin, 2006).

Finally, GOAP had computational costs that were quite important and needed to be designed around as evidenced by the decision to use a fixed size array for the world state. These costs were reduced to be manageable by limiting the scope of planning, the number of NPCs and reducing the cost of running A*. However, these optimizations are somewhat specific to this project and are not guaranteed for all GOAP implementations. In the rest of this chapter, this analysis will see how these issues are confronted in other games and projects.

Overall, F.E.A.R. shows well the advantages of using GOAP while managing the weaknesses of the techniques by using other approaches and optimizations. GOAP reduced the manpower needed to define complex agents and provided greater variety of behavior than what was offered in other games at the time. These observations will form a basis for evaluating the simpler GOAP implementation developed later in this thesis.

A look at Shadow of Mordor

Middle-earth: Shadow of Mordor is an action-adventure game released in 2014 and developed by the same company as F.E.A.R.: Monolith Productions.

Shadow of Mordor starts with the same GOAP implementation as described in F.E.A.R. The similarities will not be enumerated for brevity as the use case is mainly the same, GOAP drive the opposing agents. This analysis will concentrate on what differs in implementation or impact.

The three-state FSM layer above the GOAP is gone, its functionalities such as handling animations are kept separate from the planning. (Higley Peter, GDC AI Summit, 2015)

Shadow of Mordor has more orcs acting and planning than F.E.A.R. had clone soldiers, as such the demand on efficiency is greater. Plans are kept short by relying more on low level systems to perform actions that are not strictly necessary for the planning (Higley, 2015). For example: sheathing and unsheathing a weapon are necessary steps for attacking however they do not need to be planned for and can be played as animation on demand when attacking or moving depending on the nature of the current goal. Similarly attacking does not necessitate to move toward the player, if the plan decided to attack with a melee weapon, it is a given that the orc should go in melee range to perform it. These sorts of optimisations have a drastic impact on performances as Shadow of Mordor produces 10x more plans in the same amount of time as F.E.A.R. (Jacopin Eric, GDC AI Summit, 2015)

Goals are driven by multiple systems promoting their goals with various priorities. Among them is an FSM that describe the awareness level of an orc changing the set of

available goals and ensuring that only a limited number of goals are considered at a time.

Among the high-level systems that drive goals is the investigation system. Orcs register their interest to the investigation based on clues found of the player's presence. The system then assigns roles with associated goals to the registered agents who will then perform planning for their role (Higley, 2015). In F.E.A.R. the squad system is constantly an authority on the goals of the agents, whereas in Shadow of Mordor, orcs receive goals from the investigation when they register interest. This allows orc to roam as semi-autonomous agents until the narrative demands of the game make them transition to other behaviors. During an encounter with the player, this system allows the agents to specialise taking on unique and identifiable rolls such as investigation leader (Higley, 2015).

Compared to F.E.A.R., Shadow of Mordor shows a greater emphasis on the separation of high-level planning and low-level execution. GOAP is kept largely the same, but actions are defined at a higher level, leaving more to be handled by specialist low-level systems. Driving the goals of GOAP is a more complex system than F.E.A.R., made up of multiple systems that interact together to direct GOAP allowing for the taking of roles and specialization of agents during an encounter.

This comparison serves to illustrate that GOAP in real applications is greatly dependent on the surrounding architecture. As GOAP is a planning algorithm that uses abstractions for actions and goals, its results are a consequence of the specific abstractions chosen.

Both games examined so far suggest that GOAP is more effective when applied to high-level actions, rather than low-level actions, resulting in shorter plans composed of more complex actions instead of longer sequences of simpler ones. This idea will be examined further by using the implementation proposed by this thesis as an example.

A look at Tomb Raider

In 2013 the adventure series Tomb Raider got a new instalment simply titled Tomb Raider. With Tomb Raider, Crystal Dynamics expands on the original GOAP formula with a big focus on the agent's use of objects.

In GOAP, actions have costs for the A* algorithm to prioritize. Tomb Raider adds additional, situational costs depending on the difficulty of performing them. For example: melee attacks costs more as the target is further away (Chris Conway, GDC AI Summit, 2015). Costs can be further modified by equipment; a heavier weapon costs more to use, generalising the procedural preconditions of F.E.A.R. to these situational costs.

Tomb Raider also adds motives; values attached to agents but kept separate to the world state. A motive influences the importance of a goal and/or unlocks actions that have certain motive values as requirement. Chris Conway gives the example of an investigation goal that Is only available while suspicion is high enough. In this example, suspicion is driven by external factors, but motives can also be changed by the agent's own actions. For example: attack can decrease the fear motive as a secondary effect which controls whether the flee gal can be pursued (Conway, 2015).

When actions use objects as part of their execution, those objects can add requirements to the action; UseObject on the DinnerTable will require the agent to have Food and Drink to use (Conway, 2015). Actions can therefore have multiple sets of requirements depending on what variation of the action is chosen. Conversely, actions can propose multiple options during the planning. The UseObject example will generate multiple possible world states dependent on the object used; one option will be proposed for each valid object found for the action. This same system is then used to handle multiple targets; Attack can propose the action to Attack(character 1) and Attack(character 2) (Conway, 2015). Therefore, an action can expose multiple options where the both the set of requirements and the effects of the action can differ.

With equipment influencing an agent's course of action, this system looks like an inversion of Shadow of Mordor's approach. In Shadow of Modor, high level system distributes roles and personalize the agents where Tomb raider assigns similar goal to its agents which will specialize based on what equipment and surrounding objects are available to them.

This is further reinforced by an additional system which monitors the success rates of goals and actions, lowering the costs of successful actions and raising them in the opposite situation. (Conway, 2015)

The last addition that will be touched on in this analysis is the incorporation of a behavior graph; a high-level structure that order and prioritises goals for agents using a behavior tree-like logic. Each agent has their own behavior graph that replaces priority values for goals (Conway, 2015). This addresses the issue of designers having a hard time expressing goal authoring to be unintuitive (Orkin, 2025, interview with Jeff Orkin, see appendix) by allowing the use of more common AI design tools such as a behavior tree.

Finally, this more complex planning system also makes it more complex for designers to use and programmers to debug. GOAP is already known for these aspects which can prove difficult (Sjöblom Marcus, Svala Rasmus et al., 2025, p.35).

The implementation of GOAP in Tomb Raider shows how the technique can be extended to create more contextual behaviors while keeping all the emergent properties of the technique. While Shadow of Mordor was used to demonstrate that the

surrounding architecture of GOAP can define the behavior, Tomb Raider demonstrates that GOAP actions and planning can be modified to the design requirements. While Tomb Raider's system has difficulties in its usage, those are not reflected by longer planning phases as it produces 5x as many plans as F.E.A.R. in the same timespan (Jacopin, 2015).

Together, these three case studies show that GOAP is flexible and why it is particularly lauded for its reusable actions and contextual planning (Orkin, 2006, p.7-10) (Pttiman, 2007, p.4). However, the practical value of this flexibility is dependent on the use case and the surrounding techniques that compensate for GOAP's weaknesses. The frequent joint usage of GOAP and FSM and behavior trees indicates that one was not found to be better than the other but capitalise on different strengths.

Expert interview: Jeff Orkin

To complement the analysis of existing GOAP implementations, an interview was conducted with Jeff Orkin, who developed the GOAP system used in *F.E.A.R.* This interview touches on the situations in which GOAP is appropriate, the difficulties associated with its implementation and its position within the video game industry.

According to Orkin, GOAP is particularly well suited to games that encourage open-ended, replayable and nonlinear gameplay. Games that have systemic designs and provide multiple opportunities to encounter agents in different contexts such as RPGs and open world gams where GOAP can surprise players with novel plans. Conversely, games that have scripted and linear designs do not benefit much from GOAP (Orkin, 2025, interview with Jeff Orkin, see appendix).

This perspective reinforces the preceding case studies. The benefits of GOAP depend not only on the planning algorithm itself, but also on the extent to which the game allows agents to combine actions in different ways.

Orkin also highlights the difficulties involved in authoring planning systems. In particular: defining goals and actions can be unintuitive for designers.

This observation qualifies one of GOAP's commonly cited advantages: separating action definitions from the planning process can make behaviors easier to extend and redefine. But it does not necessarily make the overall system easier for every member of the development team to use.

These difficulties are consistent with the trade-offs identified in the analysis of *F.E.A.R.* and *Tomb Raider*, where the separation between actions and decision-making facilitates the creation of varied behaviors but can make the desired behavior less direct to specify.

Regarding industry adoption, Orkin reports that he continues to hear about games using GOAP, including projects that had already been released and others still in

development. He also notes that Hierarchical Task Network (HTN) planning is used by several studios.

Orkin expects interest in planning-based techniques to continue growing. His observations suggest that GOAP remains relevant within the broader set of tools available to game AI developers. However, they do not establish how frequently the technique is used across the industry or whether its adoption is increasing at a measurable rate.

Overall, the interview reinforces the idea that the suitability of GOAP depends on the requirements of a game. Its ability to combine actions into context-dependent plans can be valuable in systemic environments, while the effort required to author and control planning behavior remains an important practical consideration.

4 Media Project

This project can be found at this address: https://github.com/BetterDorf/GOAP

Project scope and goals

The project part of this thesis consists of the implementation of a GOAP and its usage to investigate advantages and disadvantages of the system.

Rather than attempt to recreate the complexity of a commercial game such as F.E.A.R., this project will focus on the decision making of agents in an environment that isolates the core elements of GOAP: World-State representation, conditional Actions, Planning and graph navigation.

The implementation will prioritize clarity, modularity and measurability over visual fidelity. A graphical representation of the environment will help observe agent behaviour, while logging and performance measurements will provide data for the subsequent analysis.

The main objective of this project is to implement an agent capable of making a valid plan and executing it given a set of actions and a goal. The evaluation will pertain to the resulting behavior and the computational cost of the planning phase. Finally, the versality of the system will be evaluating the production cost of changing the sets of actions and other design decisions.

Software used

This project is made using the game engine **Unity** 6.6 which provides graphics implementation, game logic and C# libraries.

Visual Studio Community Edition 2022 is the Integrated Development Environment used for editing and writing the code which is in C#. An IDE, or Integrated Development Environment, is a software application that provides tools for writing code.

Detailed project plan

Due to the compartmentalized nature of the this project and GOAP as a whole, features will be implemented separately one by one.

Simulation description

This project presents a scenario of an agent that will attack and defeat another character. This action takes place within an arena with a discrete amount of positions.

The available set of actions will represent attacks and other combat-related options such as moving and resting. Agents use their defined goal to generate a plan that satisfies it and execute it. Action planning and execution happens in a turn-by-turn structure.

Agents hold two important values, health and energy. The latter of which can be consumed by performed attacks.

Step 1: Implementing a GOAP planner

The core planning system is first implemented first in a vacuum and tested independently of the rest.

It follows the described GOAP definition in the state of the art section of this work but deviates in the specific algorithm used for navigating the graph of world-states. While F.E.A.R. uses A* (Orkin Jeff, 2006), others recommend using a variation of this algorithm called Iterative Deepening A* (IDA*) and using it in reverse, from the goal to the starting world-state (then called Reverse IDA*) (Millington Ian, 2009)

Tests and generated graphs are used to ensure that the plans created are valid where applicable.

Step 2: Environment

A simulation world is created to give context for the planner with an arena consisting of discrete tiles and characters for the agent to control.

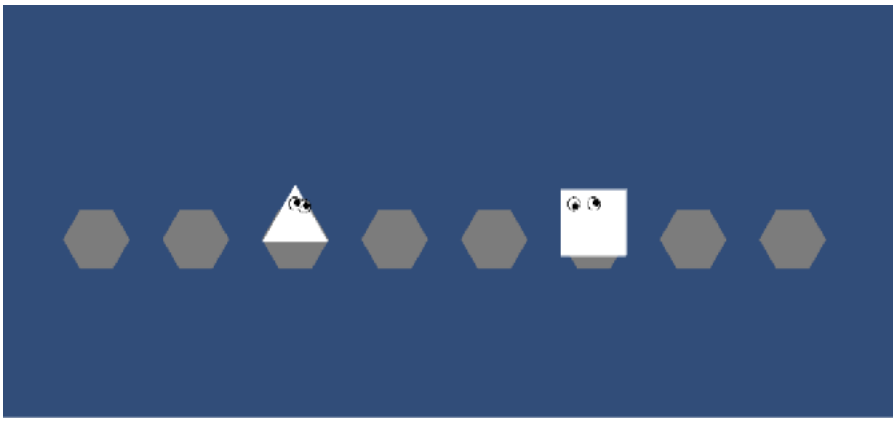


Figure 3 Illustration of a minimalist representation of the simulated world by the author in the Unity Project (2026)

A game director handles the turn structure of the characters.

Step 3: Implementation of actions and execution

Agents extract the necessary actions from the planner to perform them in the simulated world.

The implementations will be tested to identify functional errors and ensure that the measured results reflect the intended behavior.

Step 4: Conduct experiments

Scenarios of varying conditions are tested. The behavioral changes of the agent are noted and measures are taken of the various values such as computation time, memory usage and plan length.

Step 5: Analysis

Results are weighed withing the context of the research question, limitations are discussed and conclusions are drawn on the merits and trade-offs of GOAP.

Project creation

Using Unity Engine 6.6 (6000.6.0f1), a new project is created. The 2d universal template is chosen to have a simple and clean environment.

All code used is produced by the author. While no packages nor external libraries where used (excluding those that come with DotNet 9.0 and Unity 6.6), inspiration was drawn from previous GOAP implementations such as **GOAP for Unity** (Klooster Peter, 2016, code available at https://github.com/crashkonijn/GOAP) and other projects that do not make use of unity (Peikidis Panagiotis, 2010) (Stolk Abraham, 2012).

Graphic assets are geometric shapes available within the Unity Editor.

Overview of project implementation

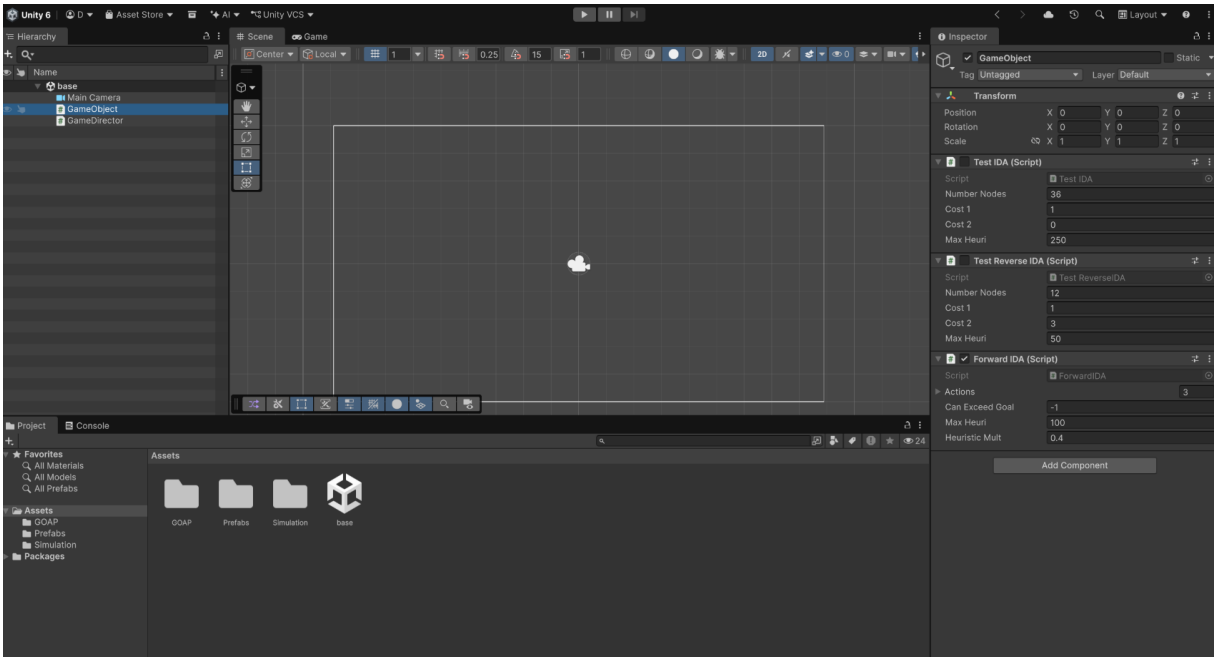


Figure 4 standard view of the Unity editor showing this project's scene and folders (2026)

In the above figure is a view of the Unity editor used in this project. Scripts are sorted in folders based on the system they represent.

GOAP contains all scripts related to the GOAP system which include definitions for GOAPactions, WorldState and planner variations.

Prefabs contains all objects used during the simulation. They are made entirely withing the editor and may be created at runtime using instantiation.

Simulation contains the scripts used to run the simulation that are separate from the GOAP logic. This separation allows for another kind of ai to be used within the context of the simulation.

GOAP WorldState

The World-State is the central structure which holds the data used during planning. Goals are expressed using the wanted WorldState as well as Action's effects and conditions.

In this implementation, the world-state is defined in WorldState.cs and is a class providing utilities wrapped around a dictionary data-structure.

Dictionaries are the standard data structure for GOAP's world-state (Millington Ian 2006, p.394) which are useful because of their access time and simple value comparisons.

In this project, the dictionary used takes the name of the value as a key for ease of use and an integer as the value. The integer value can represents properties and amounts (health, damage, distance in a discrete world, energy, etc…) or represent and id of an item stored somewhere else.

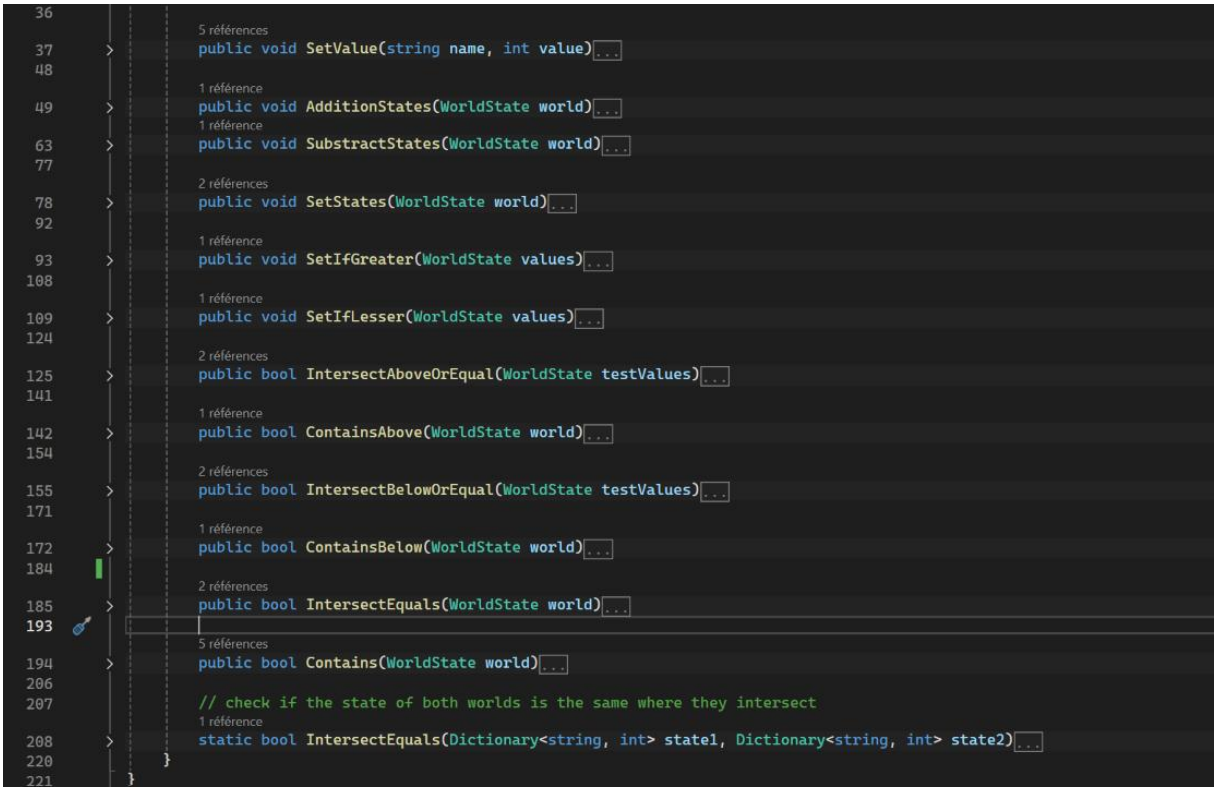


Figure 5 methods implemented within the WorldState class to help manipulate states(2026)

Methods implemented in the WorldState class define how WorldStates can be combined and compared. AdditionStates(WorldState world) adds all values of one world to another.

```
public void AdditionStates(WorldState world)
{
    foreach (var pair in world.state)
    {
        if (state.ContainsKey(pair.Key))
        {
            state[pair.Key] += pair.Value;
        }
        else
        {
            state.Add(pair.Key, pair.Value);
        }
    }
}
```

Figure 6 the AdditionStates method which takes all values of the given world and adds it to the one from which the method is called from (2026)

A distinction is made between the Contains comparisons (ContainsAbove(…), ContainsBelow(…), Contains(…)) and Interesect. The firsts verify that all values in the

world passed as parameter are present and have the correct relationship, the latters only check values that are contained in both WorldStates.

This allows defining goals as the minimum conditions that must be satisified for a given worldState to be valid.

GOAP Actions

```
public class Action
{
    [SerializeField]
    WorldState conditions;
    [SerializeField]
    WorldState aboveConds;
    [SerializeField]
    WorldState belowConds;
    [SerializeField]
    WorldState additiveEffects;
    [SerializeField]
    WorldState setEffects;

    public float cost;
    public int actionRef;
    ...
}
```

Figure 7 Partial definition of actions in this project (2026)

Actions are the building block of GOAP. They are defined using multiple WorldStates to describe when they can be performed and what effect they have.

The cost value is used during planning and is the cost of using this action as an edge between WorldStates which is compared to the heuristic.

GOAP Planner

As discussed, planning is done through the navigation of the space of possible worldStates using a pathfinding algorithm. While A* was used often, previously cited works propose that IDA* is more efficient for this purpose and is the algorithm that is used in this implementation.

A* is an algorithm that must have a starting node and an end node and will find the optimal path to reach it. It does this by considering all nodes neighboring the current node and assign to each a cost associated with reaching them (distance in pathfinding application, a resource of the agent and/or time cost for most GOAP applications). The node that is both least expensive to reach and closest to the goal is selected. This process is aided by a heuristic which evaluates the remaining distance with an approximate value. Then nodes are expanded with the same procedure choosing each time the node which is has the lowest value of cost + heuristic. So long as the heuristic

underestimate the cost of reaching the goal, the resulting path guarantees to be optimal which means in this context to have the lowest cost.

However, while A* is known to be a fast and versatile algorithm widely used in the industry, it can be very large in memory as it keeps all found nodes in memory at once. In GOAP, the graph is constructed as it is explored and is not guaranteed to be finite which is why IDA* is used in its place as it has a similar average time to find a path (but a much worse worst-case of $\Omega(n^2)$)and a lower space complexity (Millington Ian, 2009).

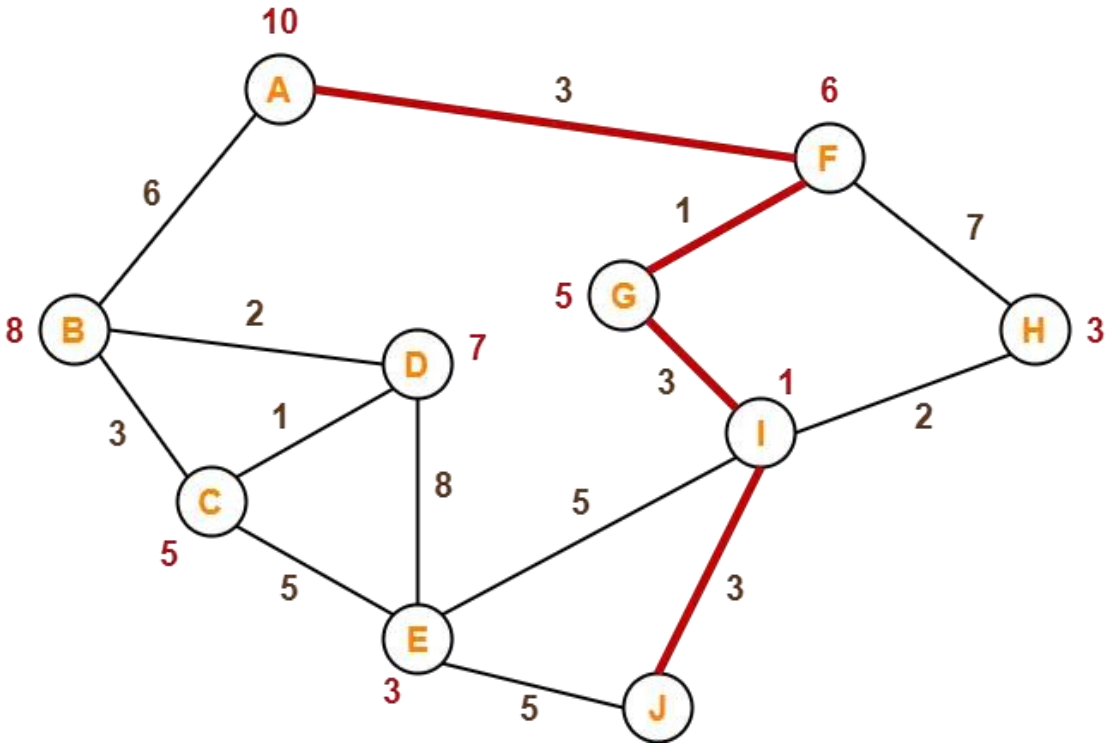


Figure 8 representation of a path chosen from J to A using A* (Representation by GateVidyalay at https://www.gatevidyalay.com/wp-content/uploads/2020/01/A-Algorithm-Problem-02-Solution.png) (2019)

Iterative Deepening A* (IDA*) is a depth-first search that takes the idea of heuristic from A* to cut down on the time spent exploring unpromising branches. IDA* does not retain branches other than the current one which both leads to less memory usage and multiple visits of the same nodes. In this recursive search, each time the exploration exceeds the projected cost of the heuristic from the current node to the end node, it is restarted from the previous node with an adjusted expected cost.

In this project, IDA* was implemented following the pseudocode available on Wikipedia at https://en.wikipedia.org/wiki/Iterative_deepening_A*.

```
public bool Search(List<GOAPNode> path, float costG, float bound, out float newBound, WorldState goal)
{
    GOAPNode node = path[path.Count - 1];

    newBound = costG + Heuristic(node);

    if (TestGoal(node, goal))
    {
        return true;
    }
    if (newBound > bound)
    {
        return false;
    }

    float min = Mathf.Infinity;
    foreach (var succ in ExpandSuccessors(node))
    {
        if (IsStateOnPath(succ, path))
        {
            continue;
        }

        path.Add(succ);

        if (Search(path, costG + actions[succ.ActionId].cost, bound, out float successorNewBound, goal))
            return true;

        if (successorNewBound < min)
            min = successorNewBound;

        path.Remove(succ);
    }

    newBound = min;

    return false;
}
```

Figure 9 Implementation of IDA* recursive Search method which forms the bulk of the logic with the ExpandSuccessor method (2026)

Reverse IDA* is a variation on IDA* where exploration is started from the goal and a path is found to the starting node. This is strictly equivalent for most applications but reduces significantly the searched space for GOAP applications. The efficiency comes from the actions that have conditions.

Let us consider a simple situation: Two nodes Start and Goal, Action A which is efficient to go from node Start to node Goal but with conditions that are not satisfied by node Start, an action B that solves A's conditions but do not approach Goal and an action C that approaches the goal slightly. In this situation IDA* will explore using action C multiple times before exploring B and finding that A can be performed to solve the situation.

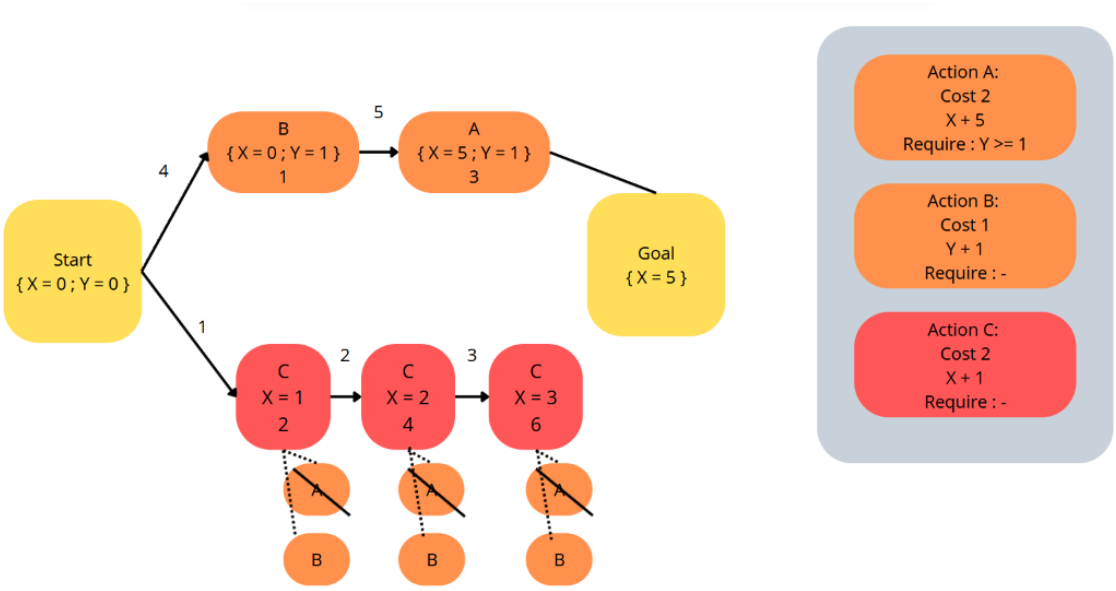


Figure 10 Forward IDA* would consider action C multiple times before finding the conditions to use A (2026)

In contrast, Reverse IDA* would find that A is the most efficient action and solve for its conditions which is often faster to produce the same plan for GOAP.

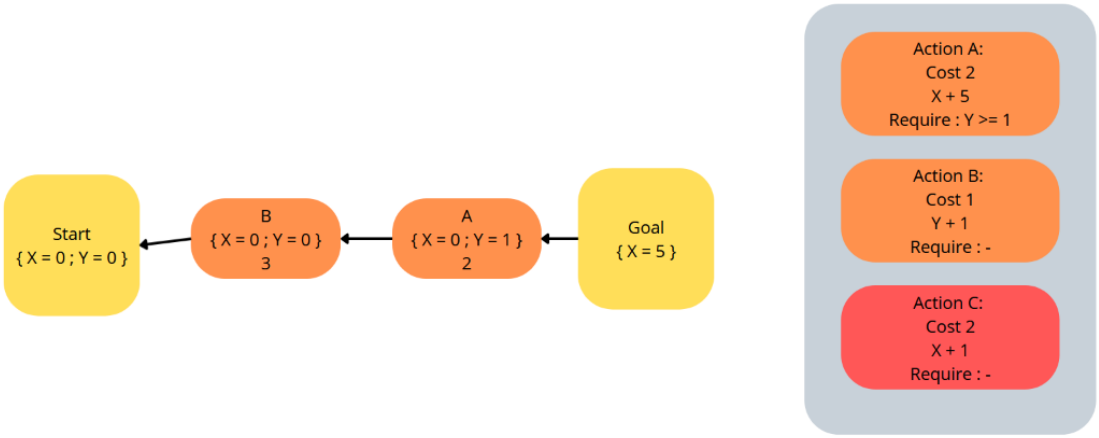


Figure 11 Reverse IDA* finds the optimal path faster (2026)

The planner is the combination of the previously described systems within the pathfinding algorithms.

For this project, both IDA* and Reverse IDA* are tested to see if the gain of Reverse IDA* are universal and/or if it sacrifices some usability.

IDA* Graph

This special version of the planner is used without an agent and served to debug and validate the plans generated.

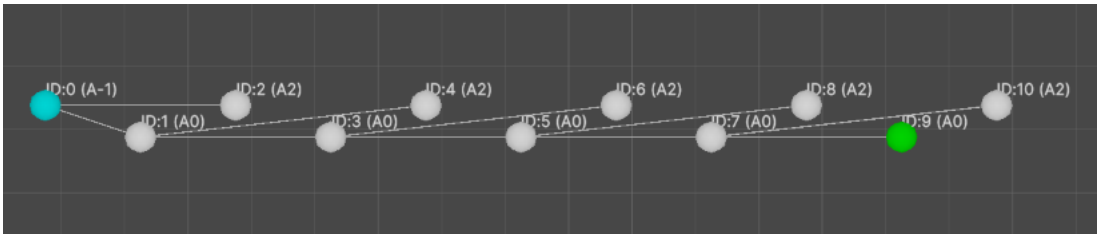


Figure 12 A valid plan was generated and the goal state reached (2026)

EnemyAI

The planner is simply a tool for agents to use. EnemyAI.cs implements one such agent. At the start of the scene, the agent supplies the associated planner with the available set of actions, the desired goal and the current worldState to start from. It then executes the given plan.

EnemyAI also has an FSM (Finite State Machine) to replace the planner, providing a comparison point.

Game Director

Defines the bound of the simulation and coordinate agents for when they plan and act.

This class serves mostly a role of convenience to change quickly the initial setup of a test.

Testing Protocol

The tests will compare the GOAP system using forward and reverse IDA* with a simple FSM.

Tests will seek to demonstrate the trade-offs of GOAP by evaluating multiple factors.

Time

Using the profiler that comes with Unity, the execution time to find an action in each situation will be measured and compared to the others.

Memory usage

Since FSM does not allocate memory to decide on an action and both IDA* and Reverse IDA* are recursive method occupying the same space in the stack, the most valuable comparison point is between the number of nodes in memory at the end of a plan.

Behavior

The author will analyse how the planning of actions affected the chose actions and what was necessary to achieve similar behavior using another technique.

Versatility

One of the key points of GOAP is its ability to function given well-defined contexts without a need for refactoring. Tests will be done in multiple scenarios and it will be noted how many and what changes where necessary for FSM and GOAP.

Testing

To gather data, multiple situations are tested for each method. Unless specified, each use the following action set:

Action name	Time to perform	Energy cost	Damage	Effect
Move	1	0	0	Move 1 closer to enemy
Rest	1	Gain 2	0	Restore energy
Punch	1	1	1	Only if distance is 1
Kick	2	3	4	Only if distance is 1
Shoot	1	2	1	Only if distance is greater than 1

Where the heuristic for planning is the time taken and each action that ask for energy cannot be performed without.

The StateMachine is simulated by considering a tired state dependent on the energy level. The actions available first depend on what energy the character has then are decided based on distance.

Unless specified otherwise the following are true: each character starts with a distance of 3 to the other in an arena of spaces; GOAP implementations plan again every turn; the heuristic distance for GOAP is 1 is equal to the difference of all values between current and goal states; the agent starts with 5 health and 5 energy.

Single Action

Setup

In this trivial scenario, one agent fights a dummy. The dummy has 1 hp.

Results

The methods behaved the same, electing to shoot the enemy, using 0.17 ms for FSM and 0.4 ms for IDA*.

Multiple Actions

Setup

Same as the previous scenario but the dummy has 5 hp and can't be beaten in a single blow.

Results

FSM

It takes 7 turns to beat the dummy using the following sequence:

Shoot -> Shoot -> Rest -> Shoot -> Shoot -> Rest -> Shoot

And took 0.5ms on average

*IDA**

A more efficient path was found by IDA* in 4 actions:

Move -> Kick -> Wait (due to kick taking 2 turns) -> punch

Interestingly, the first frame is planned in 4 ms while the rest averages around 0.2 for a total average of 1 ms

IDA* explored a total of 23 nodes during its first and longest plan.

Long plan

Setup

The dummy has been set to 50 hp and the characters start with a spacing of 5.

Results

FSM

Since the initial conditions are similar for the FSM, it applies the same pattern, shoot, shoot, rest, etc... It takes 98 turn to defeat the opponent but does so while being as fast as previously averaging 0.2ms each turn.

*IDA**

Playing the plan took 46 turns with an average of 5.25 ms to calculate a plan. However, if the agent only uses its first plan and does not recalculate it, its average drops to negligible values but this optimization only works if the situations doesn't change. It is likely that in real game applications a middle ground would be best using conditions on actions to check when the plan becomes invalidated.

The found plan consisted of moving towards the target and only kicking and alternating with rests.

The maximum number of nodes explored was 1575 which is 68.47 times more than the previous situation whose plan took 11.5 times less steps to accomplish

Change of situation

Setup

Same as multiple actions but the set of actions has been changed thusly: Kick deals 8 damage, punch restore 1 energy, shoot cost 2 more and deals 1 more. Then it is tried against the dummy that has 50 hp

Results

FSM

The result is similar in both situations. The FSM shoots and rest as it did previously. Time to decide an action was 0.21ms on average.

The implementation necessitated to change the actions themselves and the threshold to change states since those depend directly on the actions.

*IDA**

In the situation with 5 hp, the planner opts to shoot -> rest -> shoot. However, in the situation with 50 hp it decides that shooting once is enough and then approach the enemy and repeat the pattern of punching to recover energy to then spend with a kick.

This new behavior was not explicitly put into the agent, only the situation and actions available changed. In particular, the agent chose a different tactic when its opponent had a lower amount of health which shows adaptability so long as the new situation can be expressed in a world state.

The time to plan was the same as without the situation change where the 5hp case averaged 0.97ms and the 50hp one averaged 5.4ms

Analysis of results

The experiments compare a finite-state machine (FSM) with a GOAP implementation using IDA* as its planning algorithm. Both systems operate in the same simplified combat environment and use the same actions, but differ in how they select those actions. The FSM relies on explicitly defined rules based on the agent's energy and the distance to its opponent, whereas GOAP generates plans by evaluating action costs, preconditions and effects in relation to a goal.

The results are here analyzed in terms of computational performance, desirability of the resulting behavior and the ability to adapt to changes in the environment or action set.

In all scenarios, IDA* took much longer during the first turns to process a plan. This illustrates the additional overhead and cost introduced by what is a typically costly operation in videogames : pathfinding. However, as for traditional pathfinding, the cost goes down significantly if allowed to keep the same plan for longer. Replanning is among the most important performance lost associated with GOAP (Higley Peter. Jacopin Eric, 2015).

The first scenario demonstrates that a complex structure such as GOAP is not very interesting in trivial situations.

When the enemy had its life increased, GOAP was more useful as it could plan sequences. FSM only continues to shoot as it was the behavior the author planned in the first place, which did work but took longer. IDA*, by contrast, finds a plan involving movement, a kick and a punch. Although the kick requires an additional turn to complete, this sequence defeats the opponent in fewer actions and fewer turns than the FSM's strategy.

The difference illustrates one of the principal advantages of GOAP. The planner evaluates the consequences of different action sequences and can select combinations that are not explicitly defined as a strategy. Provided that the relevant actions and their effects are represented correctly, the planner can discover a more effective sequence without requiring the developer to specify that sequence in advance.

This is even clearer where the action set was changed. GOAP could use the new paradigm that came with the punch action allowing to restore energy, finding a new optimal path exploiting it. FSM had to be explicitly changed if the designer wanted it to use the new strategy.

Then for efficiency, IDA* had at worse to visit 1575 to find a plan of length 46. Although this means that at most 46 nodes were in memory at once the performance cost scaled roughly to second power of the plan’s length.

Conclusion

Across the four scenarios, the FSM consistently requires less computation per decision, while GOAP demonstrates a greater ability to combine actions into efficient, context-dependent strategies.

These differences are most significative when the planner must find solution long in the future increasing the cost. But also, when the situations are varied it showcases the different strategies that GOAP can come up with without the need to redefine behaviors.

Finally, these results show that GOAP can be used with great effect given an appropriate surrounding architecture and/or use case to direct its goals and limit its planning cost.

5 Quantitative analysis

Expected results

The author believed that GOAP would be harder to implement and be significantly longer to execute while also using much more memory than FSM. However, the author thought that those tradeoffs were justified for the additional versatility of the system.

Values Observed

Situation	FSM average decision time	IDA* average decision time	IDA* plan length	Observation
Single Action	0.17 ms	0.4 ms	1	Both chose Shoot
Multiple Actions	0.5 ms	1 ms	4	IDA* found a faster solution
Long Plan	0.2 ms	5.25 ms	46	IDA* explored a great number of extraneous nodes
Changed Action Set	0.21 ms	5.4 ms	18	IDA* changes its strategy automatically

Comparing the results, the situation is as expected, GOAP spends more computation to find a more efficient strategy, while the FSM follows a cheaper, predetermined approach.

Limitations

The experiments provide a basis to compare FSM and GOAP decision-making, but several limitations must be considered.

Firstly, the FSM and GOAP implementations do not have equivalent strategic capabilities. The FSM uses explicitly defined decision rules based on energy and distance, whereas GOAP evaluates sequences of actions using their costs and effects. While it is always possible to replicate a behavior using an FSM, it requires to be designed specifically for that situation. Consequently, some of the difference in behavioural effectiveness results from the specific strategies implemented rather than from an inherent limitation of finite-state machines.

A more sophisticated FSM could incorporate additional transitions and decision rules to produce more effective behaviour, at the cost of additional implementation and maintenance effort which would magnify the observed cost of re-designing systems.

Secondly, the simulated environment is deliberately simplified. The agents have a limited set of actions, a small number of state variables and a single opponent. Real game applications typically involve more elements and moving situations, not the least of which the player. While solutions exist for those situations as where seen in the Shadow of Mordor case study for implementing systems that organize GOAP agents and allow for modular actions for better adaptability, those features where outside the scope of this project.

Thirdly, computational measurements are dependent on the implementation and the environment in which the tests were performed. Planning time can vary according to the number of states explored and most crucially, according to implementation of the data structure. As this thesis only compared the space efficiency of an IDA* implementation against itself in various situations, optimizing for space in the worldstate representation was not a focus.

Finally, the evaluation of authoring and redesigning complexity is primarily qualitative. The experiments show that the FSM requires modifications to its decision logic when the purposes of actions changes. GOAP can select new combinations of existing actions to adapt without explicitly needing a redesign. However, this does not directly quantify the time required to implement or debug each system. A controlled authoring experiment would be needed to measure this aspect more rigorously.

Despite these limitations, the experiments provide a useful illustration of the practical trade-offs between explicit decision logic and search-based planning. They demonstrate how the choice of technique affects the behaviour produced, the computation required and the effort involved in adapting an agent to new situations. Further, the test showed the emergent properties of a GOAP agent to adapt.

6 Conclusion

This thesis compared the tradeoffs of Goal-Oriented Action Planning with other common game AI systems. It explored how the challenges inherent to GOAP were approached in real cases.

Each time, GOAP proved to be a useful tool but was wielded with additional structures to direct its goal for more dynamic experiences. Multiple systems can be used conjointly with GOAP, even GOAP itself to form a Layered GOAP (LGOAP) where, similarly to F.E.A.R. an AI system coordinate GOAP agents, however this system is also a GOAP agent (Maggiore Giuseppe, 2015).

The practical implementation and experiments provided a simplified comparison between GOAP using IDA* and a finite-state machine. The results showed that the FSM made decisions more quickly in all scenarios with recorded timings. However, the GOAP implementation found more efficient action sequences in the multiple-action and long-plan scenarios, reducing the number of turns required to defeat the opponent. This research did not establish a superiority between GOAP and FSM as simple AI agents are likely better off not demanding high computational costs.

These properties make GOAP a likely candidate for large games with a particular emphasis on NPCs (Orkin, interview, 2025).

Overall, GOAP is a delightful technique that requires more overhead than most techniques to implement but behaves nearly autonomously afterwards.

While what techniques will be used by the industry in the future is quite uncertain given the constantly evolving field of AI research including generative AIs, it is likely that GOAP and techniques that evolve from it are optimal for certain games and experiences.

Bibliography

Alabi Moses, 2025, AI in Gaming: Procedural Content Generation, NPC Behavior, and Real-Time Strategy, PDF, Accessed on: Oct 09 2026, Available at: <https://www.researchgate.net/publication/390557730_AI_in_Gaming_Procedural_Content_Generation_NPC_Behavior_and_Real-Time_Strategy>

Anon account bigdog, 2006,FEAR SDK, found on GameBanana, Accessed on: Oct 04 2026, Available at: <https://gamebanana.com/tools/397>

Champandard Alex J., 2007, Assaulting F.E.A.R.'s AI: 29 Tricks to Arm Your Game, Blogpost, Accessed on: Oct 06 2026, Available at: <https://web.archive.org/web/20080124172408/http://aigamedev.com/reviews/fear-ai>

Champandard Alex J.,2008 , Goal-Oriented Action Planning, Blogpost, Accessed on: Oct 06 2026, Available at: <https://web.archive.org/web/20160313121023/http://aigamedev.com/premium/reports/goal-oriented-action-planning/>

David Pittman, 2007, Pratical development of goal-oriented action planning AI, PDF, Accessed on: Sep 28 2026, Available at: <https://autodiscover.dphrygian.com/bin/david-pittman-goap-masters-thesis.pdf>

Doris Ken and Silvia David, 2007, Improved Missile Route Planning and Targeting using Game-Based Computational Intelligence, PDF, Accessed on: Oct 05 2026, Available at: <https://vigir.missouri.edu/~gdesouza/Research/Conference_Cds/IEEE_SSCI_2007/CI%20in%20Security%20and%20Defense%20Applications%20-%20CISDA%202007/data/papers/CISDA/S004P003.pdf>

Edmund McMillen, 2025, Mewgenics Live Stream! Feat. Tyler (Edited), Youtube Video, Accessed on: Oct 09 2026, Available at: <https://www.youtube.com/watch?v=rVNQ4jh4lTw>

Humphreys Troy, DATE, Exploring HTN Planners through Example, PDF, Accessed on: Sep 30 2026, Available at: <https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter12_Exploring_HTN_Planners_through_Example.pdf>

Iovino Matteo, Scukins Edvards, Styrud Jonathan, Ögren Petter, Smith Christian, 2022, A survey of Behavior Trees in robotics and AI, PDF, Accessed on: Oct 09 2026, Available at:

<https://www.sciencedirect.com/science/article/pii/S0921889022000513>

Kelly John-Paul, Botea Adi, Koenig Sven, 2007, Planning with Hierarchical Task Networks in Video Games, PDF, Accessed on: Oct 08 2026, Available at: <https://icaps07-satellite.icaps-conference.org/workshop8/Planning%20with%20Hierarchical%20Task%20Networks%20in%20Video%20Games.pdf>

Kelly John-Paul, Botea Adi, Koenig Sven, 2008, Offline Planning with Hierarchical Task Networks in Video Games, PDF, Accessed on: Oct 08 2026, Available at: <https://ojs.aaai.org/index.php/AIIDE/article/view/18673>

Long Edmund, 2007, Enhanced NPC Behaviour using Goal Oriented Action Planning, PDF, Accessed on: Oct 02 2026, Available at: <https://web.archive.org/web/20180107033603/http://www.edmundlong.com/Projects/Masters_EnhancedBehaviourGOAP_EddieLong.pdf>

Maggiore Giuseppe, Santos Carlos, Dini Dino, Peters Frank, Bouwknegt Hans, Spronck Pieter, 2013, LGOAP: adaptive layered planning for real-time videogames, PDF, Accessed on: Oct 03 2026, Available at: <https://www.spronck.net/pubs/CIG2013Maggiore.pdf> (Lgoap)

Millington Ian, 2009, Artificial Intelligence for Games, second edition, Publisher: CRC Press

Neufeld, X., Mostaghim, S., & Brand, S. (2018). A Hybrid Approach to Planning and Execution in Dynamic Environments Through Hierarchical Task Networks and Behavior Trees. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, *14*(1), 201–207., Accessed on: Oct 07 2026, Available at: <https://doi.org/10.1609/aiide.v14i1.13044>

Ocampo Jason, 2005, F.E.A.R review, blogpost, Accessed on: Oct 09 2026, Available at: <https://www.gamespot.com/reviews/fear-review/1900-6135744/>

Orkin Jeff, 2004, Symbolic Representation of Game World State: Toward Real-Time Planning in Games, PDF, Accessed on: Oct 03 2026, Available at: <https://cdn.aaai.org/Workshops/2004/WS-04-04/WS04-04-006.pdf>

Orkin Jeff, 2008, “Applying Goal-Oriented Action Planning to Games.” AI Game Programming Wisdom 2, Charles River Media, Accessed on: Oct 03 2026, Available at: <https://drive.google.com/file/d/0B3-H32fPH4lgcWl1alc3cWpZaVk/view?resourcekey=0-wsaTAaiNAi9yUpeBd0BFIQ>

Chris Conway, Peter Higley, Eric Jacopin, 2015, Goal-Oriented Action Planning: Ten Years Old and No Fear!, PDF, Accessed on: Oct 03 2026, Available at: <https://www.gdcvault.com/play/1022019/Goal-Oriented-Action-Planning-Ten>

Orkin Jeff, 2006, Three States and a Plan: The A.I. of F.E.A.R., PDF, Accessed on: Oct 03 2026, Available at: <https://www.gamedevs.org/uploads/three-states-plan-ai-of-fear.pdf>

Orkin Jeff, Smith Tynan , Reckman Hilke, and Roy Deb, 2010, Semi-automatic task recognition for interactive narratives with EAT & RUN, In Proceedings of the Intelligent

Narrative Technologies III Workshop (INT3 '10). Association for Computing Machinery, New York, NY, USA, Article 3, 1–8, Accessed on: Oct 03 2026, Available at: <https://doi.org/10.1145/1822309.1822312>

Peikidis Panagiotis, 2010, StarPlanner Demonstrating the use of planning in a video game, PDF, Accessed on: Oct 05 2026, Available at: <https://pekalicious.github.io/StarPlanner/report/peikidis.pdf>

Stolk Abraham, 2012, General Purpose GOAP, Code repository, Accessed on: Oct 07 2026, Available at: <https://github.com/stolk/GPGOAP>

Klooster Peter, 2016, GOAP for Unity, Code repository, Accessed on: April 07 2025, Available at: < https://github.com/crashkonijn/GOAP>

Pervolarakis Zacharias, 2023, Autonomia: A knowledge-based framework for realistic agent behaviours in dynamic video game environments, Masters Thesis, PDF, Accessed on: Sep 29 2026, Available at: <https://elocus.lib.uoc.gr/dlib/3/3/2/attached-metadata-dlib-1701959625-154345-17277/Pervolarakis_Zaxarias_Msc_Thesis.pdf>
Shafer Jon, 2013, Revisiting the Design of Civ 5, BlogPost, Accessed on: Oct 09 2026, Available at: <https://www.gamedeveloper.com/design/revisiting-the-design-of-civ-5>

Sjöblom Marcus and Svala Rasmus, 2025, Multi-Agent Performance Using Goal-Oriented Action Planning (GOAP) in a Procedural Game, PDF, Accessed on: Oct 06 2026, Available at: <https://www.diva-portal.org/smash/get/diva2:1972169/FULLTEXT01.pdf>

Sun, L., Jiao, P., Xu, K., Yin, Q., & Zha, Y., 2017, Modified Adversarial Hierarchical Task Network Planning in Real-Time Strategy Games. Applied Sciences, 7(9), 872, Accessed on: Sep 28 2026, Available at: <https://doi.org/10.3390/app7090872>

Soemers D. J. N. J. and Winands M. H. M., "Hierarchical Task Network Plan Reuse for video games," *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, Santorini, Greece, 2016, pp. 1-8, doi: 10.1109/CIG.2016.7860395, Accessed on: Sep 30 2026, Available at: <https://ieeexplore.ieee.org/abstract/document/7860395>

Thompson Tommy, 2024, AI 101: How AI is Actually Used in the Video Games Industry, Blogpost, Accessed on: Oct 09 2026, Available at: <https://www.aiandgames.com/p/how-ai-is-actually-used-in-the-video>

Trivedi Bhushan, 2018, Artificial Intelligence, E Book, Accessed on: Oct 02 2026, Available at: <https://ebooks.inflibnet.ac.in/itp6/chapter/iterative-deepening-a-recursive-best-first-agents/> (IDA*)

This Is Vini , 2021, Intro to Goal Oriented Action Planning (GOAP), Youtube Video, Available at: <https://www.youtube.com/watch?v=LhnlNKWh7oc>

Iterative Deepening A* 2026 Wikipedia. Available at: https://en.wikipedia.org/wiki/Iterative_deepening_A*, Accessed on: Sep 07 2025

Ludography

Atari, 1972, Pong, Web based emulator, Publisher: Atari

Team Cherry, 2017, Hollow Knight, Windows, Publisher: Team Cherry

Monolith Production, 2005, F.E.A.R, Windows, Publisher: Vivendi Universal Games

Monolith Productions, 2014, Middle-earth: Shadow of Mordor, PS4, Publisher: Warner Bros. Interactive Entertainment[c]

Crystal Dynamics, 2013, Tomb Raider, playstation 4, Publisher: Square Enix

Edmund McMillen-Tyler Glaiel, 2026, Mewgenics, Windows, Publisher: Edmund McMillen-Tyler Glaiel

Firaxis Games, 2010, Civilisation 5, Windows, Publisher: 2K

List of Figures

Figure 1: Image recreating the graph of Three States and a Plan (2006) by Jeff Orkin in which he details the functionement of GOAP in its first commercial game. (2026)

Figure 2: Graph produced by the author to illustrate the relationship of the AI systems that govern the behavior of the agents in F.E.A.R. (2026)

Figure 3: Screenshot of the media project of the author to illustrate the simulated world in the Unity Project (2026)

Figure 4: Screenshot of the author's Unity project as viewed in the main view of the editor. (2026)

Figure 5 & 6: Snippets of code from the file WorldState.cs of the author's media project that illustrate its structure and purpose. (2026)

Figure 7: Snippet of code from the Action class in GOAPAction.cs to show how the WorldState class relates to the action class. (2026)

Figure 8: Example of a path taken by an A* algorithm to navigate a graph. Image taken from the website https://www.gatevidyalay.com/wp-content/uploads/2020/01/A-Algorithm-Problem-02-Solution.png (2026)

Figure 9: The implementation of the recursive method to search a graph using IDA* as found in the media project in the file ForwardIDA.cs and following the pseudo code described at https://en.wikipedia.org/wiki/Iterative_deepening_A* (2026)

Figure 10 & 11: Graph made by the author to demonstrate the order of node exploration from IDA* and Reverse IDA*. (2026)

Appendix

Interview with expert : Jeff Orkin

Email conversation on 1st April 2025

Questions	Réponses
In what situations would you consider using GOAP ?	Good when working with designers who are supportive of open-ended, replayable, non-linear gameplay. Does not add much to a linear scripted game. Good for open world games, RPGs, games where the design is systemic. These games provides an opportunity for GOAP to discover novel plans that delight and surprise the player
What are the difficulties when working with GOAP ?	watch video : https://www.youtube.com/watch?v=3Of2NtTYcvU (interview)
Have you heard about LGOAP ?	No. But sounds like a good idea.
Would you use LGOAP and for what situations ?	Maybe. I would need to think more about it, and have a specific problem in mind to solve.
What would be the challenges with LGOAP ?	Yes (to performance issues), could introduce performance challenges when planning multiple times in one call to the planner. And could be more difficult for designers to author planning operators. This is already a problem for planners -- designers often find authoring the goals, actions, tasks, etc to be unintuitive. It often ends up requiring the AI engineer(s) to author the planning domain content. It seems like LGOAP is hierarchical planning. This could have benefits in not needing to refine lower level plans if planning at the higher level fails. But when doing hierarchical planning, many developers will choose HTN because it is more established. However HTN is forward planning, so LGOAP might be preferable if the desire is for backward planning. It also may be possible to combine GOAP and HTN to get a different kind of hierarchical planning.
according to you and your experience, what is the perspective of the industry regarding GOAP and how do you think it will evolve in the near future ?	I think Transformers used HTN. But yes, others have used GOAP. I continue to hear of other games using GOAP that I did not know about, that have already shipped, and some still in development. HTN is used by several studios too. I have not heard of anyone using LGOAP. But it is possible that some are. I think interest in planning will continue to grow.

Keeping in mind all that we've talked about, are there important blindspots in this discussion ?	Nope, you covered it.